EXPERIMENT NO. 10

Implement Abstraction using Abstract Classes

PART B – TO BE COMPLETED BY STUDENTS

Roll No. C-60

Name: Avdhut Parish Madhavi

Class / Division: SE-C

Batch: C2

Date of Experiment _____

Date of Submission _____

Grade _____

B.1 Document Created by the Student

Create another concrete service and implement the same abstract method.

Another concrete service named PharmacyService is created by extending the abstract class HealthService. It implements the abstract method provideService(). This demonstrates how different services can share a common abstract class while providing their own implementations.

B.2 Student Program / Output

Paste/write the final program and attach or reproduce the observed output below.

```
abstract class HealthService {
   String serviceName;
   HealthService(String serviceName) {
      this.serviceName = serviceName;
   }
   abstract void provideService();

   void displayServiceName() {
      System.out.println("Service: " + serviceName);
```

```
    }
}
class LaboratoryService extends HealthService {
    LaboratoryService() {
        super("Laboratory");
    }
    @Override
    void provideService() {
        System.out.println("Laboratory service activated.");
    }
}
class PharmacyService extends HealthService {
    PharmacyService() {
        super("Pharmacy");
    }
    @Override
    void provideService() {
        System.out.println("Pharmacy service activated.");
    }
}
public class AbstractClassDemo {
    public static void main(String[] args) {
        HealthService service1 = new LaboratoryService();
        HealthService service2 = new PharmacyService();
        service1.displayServiceName();
        service1.provideService();
        service2.displayServiceName();
        service2.provideService();
    }
```

}

```
Service: Laboratory
Laboratory service activated.
Service: Pharmacy
Pharmacy service activated.
```

B.3 Observations and Learning

● Why can an abstract class not normally be instantiated?

An abstract class cannot be instantiated directly because it may contain abstract methods without implementations. A concrete subclass must implement the required abstract methods.

● Can an abstract class contain concrete methods?

Yes. An abstract class can contain both abstract methods and concrete methods with complete implementations.

● When is abstraction useful in software design?

Abstraction is useful when common features need to be defined while allowing subclasses to implement specific behavior. It improves code organization, reusability, and maintainability.

Student observation / learning:

I learned how abstract classes define common properties and methods for related services. LaboratoryService and PharmacyService implement the same abstract method differently.

B.4 Conclusion

Modular healthcare architecture is a stated syllabus context.

Student conclusion:

The experiment successfully demonstrates abstraction using an abstract class, abstract methods, concrete methods, and method overriding. Different healthcare services share a common structure while providing specialized implementations.

B.5 Questions of Curiosity

6. Why is an abstract class used?

An abstract class provides a common structure for related subclasses and allows common behavior to be reused.

7. Can an abstract class contain concrete methods?

Yes. It can contain implemented methods, constructors, variables, and abstract methods.

8. When is abstraction useful?

Abstraction is useful when a common interface or structure is needed while hiding implementation details and allowing specialized behavior in subclasses.